Búsqueda de Soluciones a partir de Algoritmos: Planificación de Tiempos Semafóricos con el Agente SemaforIA-BOG

Ana Milena Uribe
Politecnico Gran Colombiano
Inteligencia Artificial: Unidad 3, Actividad formativa
Carlos Cañon Romero
02 de Noviembre de 2026

Búsqueda de Soluciones a partir de Algoritmos: Planificación de Tiempos Semafóricos con el Agente SemaforIA-BOG

Se retoma el agente SemaforIA-BOG trabajado en la unidad anterior: un agente basado en utilidad que apoya a los operadores del Centro de Gestión de Tránsito (CGT) ajustando los semáforos de un corredor arterial de Bogotá, ciudad donde un conductor perdió cerca de 122 horas en trancones en 2022 (Redacción Bogotá, 2023). Sus sensores (cámaras, espiras y pluviómetro) le entregan la cola de cada acceso, la tasa de llegada de vehículos y si llueve; su actuador es el controlador semafórico. En esta unidad se construye su módulo de decisión: cuando una intersección queda congestionada, por ejemplo tras un aguacero, el agente debe encontrar la secuencia de tiempos de verde que la devuelve a operación normal con la menor demora total para los usuarios. Hoy esa tarea la resuelve un operador por ensayo y error o un plan de tiempos fijos; formulada como problema de búsqueda, el agente puede resolverla de forma sistemática (Cano Niño, s. f.; Russell y Norvig, 2021).

Modelo del Problema de Búsqueda

La Tabla 1 define los cinco elementos del problema. Las colas se modelan como un fluido determinista (valores esperados), el mismo supuesto del método de Webster (1958) para el cálculo de semáforos.

Tabla 1

Elementos del espacio de búsqueda

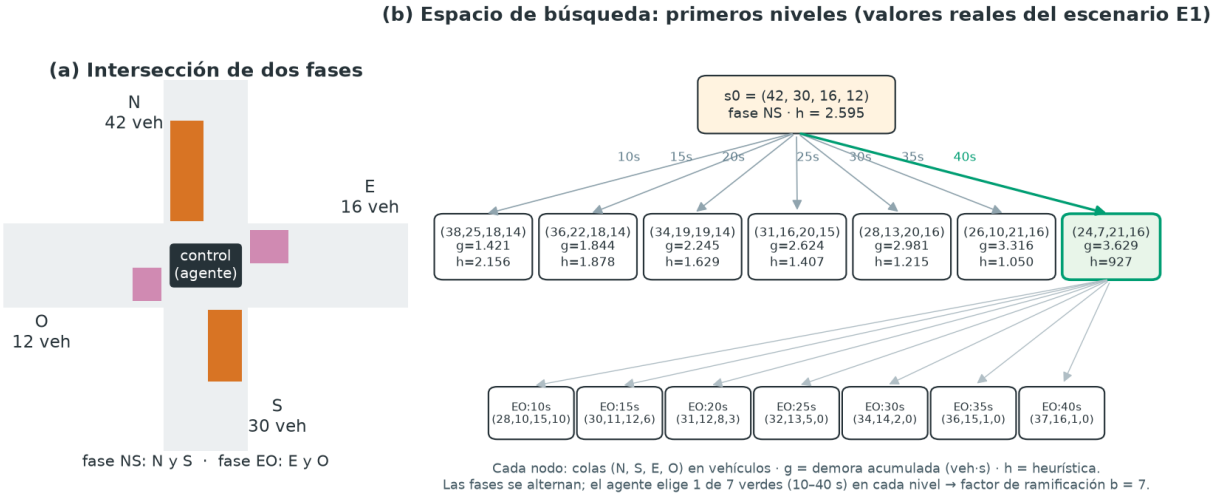
Elemento	Definición en el agente SemaforIA-BOG
Estado	$s = (q_N; q_S; q_E; q_O; f)$: cola de cada acceso en vehículos y fase f que recibe el próximo verde (NS: norte-sur; EO: este-oeste).
Estado inicial	Lo que percibe el agente. En el escenario E1: (42; 30; 16; 12; NS), con llegadas λ = (0,30; 0,20; 0,12; 0,10) veh/s y tasa de descarga s = 0,80 veh/s (0,64 con lluvia).
Acciones	Dar verde a la fase f durante g = 10, 15, …, 40 s: siete acciones. 10 s es el verde mínimo peatonal y 40 s el máximo para no dejar en rojo indefinido a los demás accesos.
Modelo de transición	Durante g s los accesos con verde bajan su cola a la tasa ($s - \lambda$) hasta vaciarse y los demás acumulan $\lambda \cdot g$; luego hay 5 s de ámbar y todo rojo en los que todos acumulan, y la fase cambia. Si una cola supera 60 veh (desborde hacia la intersección anterior), la acción no es válida.
Estado objetivo	Toda cola ≤ 8 veh: la intersección opera sin cola residual significativa.
Función de costo	Demora en vehículo-segundo: área bajo la curva de cola de los cuatro accesos durante la acción. El costo del camino es la demora total.

El problema es un grafo dirigido de estados (Figura 1). Las características que más pesan en la elección del algoritmo son: (a) los costos de las acciones no son uniformes, pues un verde de 40 s causa más demora que uno de 10 s pero despeja más, así que el plan con menos acciones no es necesariamente el más barato; (b) hay estados repetidos, porque distintos órdenes de verdes llevan a las mismas colas, lo que exige búsqueda en grafo; (c) el factor de ramificación es b = 7 y las soluciones tienen entre 5 y 11 acciones, por lo que la

frontera puede crecer como 7^d; (d) el objetivo es un conjunto de estados, no uno solo; y (e) el dominio permite construir una heurística admisible, porque se conoce la tasa máxima a la que puede bajar cada cola.

Figura 1

Representación abstracta del problema y primeros niveles del árbol de búsqueda



Nota. Las colas se expresan en vehículos (N, S, E, O); *g* = demora acumulada (veh·s) y *h* = heurística. En verde, la primera acción del plan óptimo hallado por A* y por costo uniforme.

Selección y Justificación de los Algoritmos

Se eligieron dos algoritmos no informados y dos informados para medir cuánto aporta la heurística (Tabla 2). El criterio principal es la optimalidad en costo, porque la demora es lo que sufren los ciudadanos; el secundario es la eficiencia, porque el agente debe decidir en pocos segundos con el cómputo del armario semafórico.

Tabla 2

Algoritmos seleccionados y su justificación técnica

Algoritmo	Tipo	¿Óptimo en este problema?	Papel en el estudio
Búsqueda en anchura (BFS)	No informada	No: minimiza el número de acciones, no la demora	Línea base: plan con menos cambios de fase
Costo uniforme (UCS)	No informada	Sí, con costos positivos (Dijkstra, 1959)	Referencia del óptimo y del esfuerzo sin heurística
Codiciosa del mejor primero	Informada, $f = h$	No	Muy rápida; útil si el tiempo de decisión es crítico
A*	Informada, $f = g + h$	Sí, si h es admisible (Hart et al., 1968)	Solución propuesta

Se descartaron la búsqueda en profundidad (no es óptima), la profundidad iterativa (ignora los costos, como BFS) y la bidireccional (el objetivo es un conjunto de estados y la

transición no es invertible).

Para la heurística se usó que ninguna cola puede bajar más rápido que $r_i = s - \lambda_i$ y que la búsqueda no termina hasta que cada cola llega a la meta m. Por tanto, la demora mínima que aún aportará cada acceso es el área del trapecio que baja de q_i a m a esa tasa:

$$h(n) = \Sigma\ (q_i^2 - m^2) / (2r_i), \text{ para cada acceso con } q_i > m$$

La cota ignora que las fases compiten por el verde y el tiempo perdido, así que nunca sobreestima: es admisible y además consistente. El programa lo verifica: en 100 instancias y 3.941 transiciones hubo 0 violaciones de admisibilidad y 0 de consistencia. En promedio, h del estado inicial equivale al 26 % del costo óptimo real, es decir, informa pero de forma conservadora.

Implementación en Python

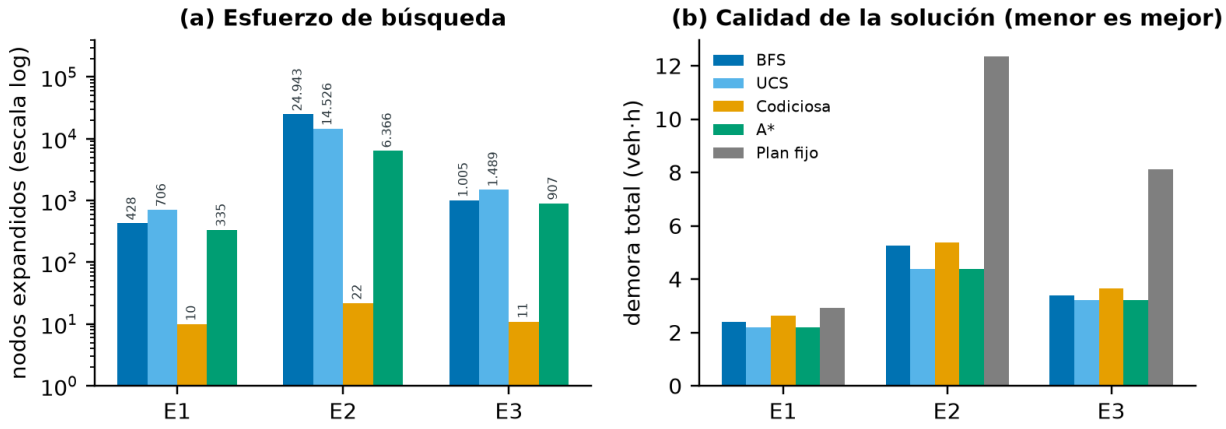
El código se organiza en módulos (Anexo C): `modelo.py` define el problema y la clase `AgenteSemaforIA` (percibir, decidir y actuar); `busqueda.py`, los cuatro algoritmos como búsqueda en grafo; `experimentos.py`, las pruebas. Las colas se guardan como enteros (centésimas de vehículo) para reconocer sin error los estados repetidos. Todo se reproduce con `python main.py`, que guarda los resultados en CSV; las figuras se hicieron con Matplotlib (Hunter, 2007). El Anexo B muestra al agente en ejecución.

Resultados y Comparación

Se evaluaron tres escenarios del corredor: E1, hora pico con vía seca; E2, el mismo estado inicial con lluvia, que hace la descarga 20 % más lenta; y E3, la salida de un evento masivo, con colas largas en los accesos este y oeste. Como línea base se simuló el plan de tiempos fijos actual (30 s NS y 20 s EO). La Figura 2 y la Tabla 3 resumen el desempeño.

Figura 2

Esfuerzo de búsqueda y calidad de la solución por escenario



Nota. En (a) el eje vertical está en escala logarítmica. Elaboración propia.

Tabla 3

Desempeño de los algoritmos en los tres escenarios

Escenario	Método	Demora (veh·h)	Brecha frente al óptimo	Acciones / duración	Nodos expandidos	Frontera máxima	Tiempo (ms)
E1. Hora pico, vía seca	BFS	2,39	9,3 %	5 / 165 s	428	859	4,2
	UCS	2,19	0,0 %	5 / 165 s	706	849	6,6
	Codiciosa	2,62	19,8 %	9 / 215 s	10	61	0,1
	A*	2,19	0,0 %	5 / 165 s	335	601	3,6
	Plan fijo	2,93	34,1 %	9 / 275 s	—	—	—
E2. Hora pico con lluvia	BFS	5,25	19,6 %	11 / 355 s	24.943	16.592	259,5
	UCS	4,39	0,0 %	11 / 355 s	14.526	11.073	140,1
	Codiciosa	5,38	22,5 %	17 / 440 s	22	133	0,3
	A*	4,39	0,0 %	11 / 355 s	6.366	5.757	63,9
	Plan fijo	12,35	181,3 %	45 / 1.355 s	—	—	—
E3. Salida de evento masivo	BFS	3,39	5,6 %	6 / 195 s	1.005	1.620	10,0
	UCS	3,21	0,0 %	6 / 195 s	1.489	2.783	15,2
	Codiciosa	3,64	13,3 %	11 / 255 s	11	67	0,1
	A*	3,21	0,0 %	6 / 195 s	907	1.452	9,6
	Plan fijo	8,12	152,6 %	20 / 600 s	—	—	—

Nota. Brecha = exceso de demora frente al plan óptimo. Los tiempos corresponden a un solo hilo de Python y pueden variar según el equipo.

Para comprobar que las diferencias no dependen de tres casos, se resolvieron 100 instancias aleatorias del mismo corredor (Tabla 4).

Tabla 4

Resumen sobre 100 instancias aleatorias

Algoritmo	Soluciones óptimas	Brecha media / máxima	Nodos expandidos (mediana)	Ramificación efectiva b^* (mediana)	Tiempo mediana / máximo (ms)
BFS	13 de 100	10,3 % / 36,3 %	302	2,86	2,9 / 1.496
UCS	100 de 100	0,0 % / 0,0 %	740	3,98	6,9 / 1.113
Codiciosa	0 de 100	20,5 % / 94,9 %	8	1,03	0,1 / 1
A*	100 de 100	0,0 % / 0,0 %	440	3,48	4,7 / 794

Nota. Instancias generadas con semilla 2026; b^* es el factor de ramificación efectivo (Russell y Norvig, 2021).

En eficiencia, la heurística sí reduce el esfuerzo: A* expandió entre 39 % y 56 % menos nodos que UCS en los escenarios, y en las instancias aleatorias la mediana de la razón A*/UCS fue 0,57. La codiciosa es la más rápida, con decenas de nodos frente a miles, porque avanza casi en línea recta hacia la meta. BFS fue el más costoso con lluvia (E2), cuyos planes son más largos.

En precisión, A* y UCS encontraron siempre el mismo costo óptimo, como predice la teoría. BFS solo fue óptimo en 13 de 100 instancias, con una brecha media de 10,3 %, porque prefiere planes con menos cambios de fase aunque generen más demora. La codiciosa fue óptima en 0 de 100 (brecha media de 20,5 %): elige el verde más largo para la fase cargada y luego necesita muchas acciones cortas para terminar.

En adecuación al problema, el plan de A* redujo la demora frente al plan fijo en 25 % (E1), 64 % (E2) y 60 % (E3). Con lluvia, el plan fijo tardó 1.355 s en despejar, frente a 355 s del plan óptimo (Anexo D), porque sus 30 s de verde apenas alcanzan para la demanda del acceso norte cuando los vehículos arrancan más despacio. A* es el algoritmo más adecuado: garantiza el óptimo y, aun en el peor caso observado, decide en una fracción del ciclo del semáforo.

Reflexión Final

La decisión más importante fue el modelado, más que el algoritmo. Definir el costo como demora, y no como número de acciones o tiempo hasta despejar, es lo que separa a BFS del óptimo y alinea al agente con la medida de desempeño definida en la unidad anterior. Representar las colas con enteros y usar búsqueda en grafo evitó expandir varias veces estados idénticos, y la restricción de desborde hizo finito el espacio. También se aprendió que una heurística admisible puede ser correcta y aun así poco informada: la nuestra recorta el esfuerzo de UCS, pero al ignorar el conflicto entre fases subestima bastante el costo real.

El modelo tiene límites: supone llegadas constantes y conocidas, por lo que en la práctica el agente debería replanificar en cada ciclo con nuevas percepciones (horizonte móvil). Como mejoras se proponen una heurística más fuerte que considere que las fases no pueden servirse a la vez; A* ponderado, para cambiar un poco de optimalidad por velocidad; coordinar varias intersecciones del corredor, lo que convierte el problema en multiagente; y validar el modelo con aforos reales y simulación de tránsito antes de llevarlo a la calle.

Referencias

Cano Niño, M. A. (s. f.). *Introducción a problemas de búsqueda* [Lectura fundamental, Módulo Inteligencia Artificial, Unidad 3]. Material institucional del curso.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*, 269–271. https://doi.org/10.1007/BF01386390

Hart, P. E., Nilsson, N. J. y Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Redacción Bogotá. (2023, 11 de enero). Bogotá, la sexta ciudad con más tráfico en el mundo, ¿cuántas horas se pierden? *El Espectador*. https://www.elespectador.com/bogota/bogota-la-sexta-ciudad-con-mas-trafico-en-el-mundo-cuantas-horas-se-pierden/

Russell, S. J. y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper N.º 39). Her Majesty's Stationery Office.

Anexo A

Plan Óptimo de A* en el Escenario con Lluvia (E2)

Paso	Acción	Colas antes (N, S, E, O)	Colas después	Demora del paso (veh·s)	*g* acumulado	*h*(estado)
1	NS 40 s	42,0; 30,0; 16,0; 12,0	29,9; 13,4; 21,4; 16,5	3.949	3.949	1.924
2	EO 30 s	29,9; 13,4; 21,4; 16,5	40,4; 20,4; 6,4; 0,8	2.515	6.464	2.706
3	NS 40 s	40,4; 20,4; 6,4; 0,8	28,3; 3,8; 11,8; 5,3	2.509	8.973	1.156
4	EO 20 s	28,3; 3,8; 11,8; 5,3	35,8; 8,8; 2,0; 0,5	1.127	10.100	1.806
5	NS 40 s	35,8; 8,8; 2,0; 0,5	23,7; 1,0; 7,4; 5,0	1.700	11.800	732
6	EO 10 s	23,7; 1,0; 7,4; 5,0	28,2; 4,0; 2,8; 0,5	512	12.312	1.075
7	NS 40 s	28,2; 4,0; 2,8; 0,5	16,1; 1,0; 8,2; 5,0	1.325	13.636	290
8	EO 10 s	16,1; 1,0; 8,2; 5,0	20,6; 4,0; 3,6; 0,5	410	14.046	530
9	NS 40 s	20,6; 4,0; 3,6; 0,5	8,5; 1,0; 9,0; 5,0	1.019	15.065	28
10	EO 10 s	8,5; 1,0; 9,0; 5,0	13,0; 4,0; 4,4; 0,5	308	15.372	154
11	NS 20 s	13,0; 4,0; 4,4; 0,5	7,7; 1,0; 7,4; 3,0	439	15.811	0

Nota. Colas en vehículos; demora y *g* en vehículo-segundo.

Anexo B

Ejecución del Agente: Ciclo Percibir, Decidir y Actuar

Salida del programa cuando el agente recibe las lecturas de sus sensores en el escenario con lluvia, planifica con A* y envía las órdenes al controlador semafórico.

```
>> PERCIBIR    {'colas_veh': (42, 30, 16, 12), 'llegadas_veh_s': (0.3, 0.2, 0.12, 0.1), 'lluvia_mm_h': 8.5}
   lluvia > 2 mm/h -> tasa de descarga 0.64 veh/s
   estado inicial [N=42.0, S=30.0, E=16.0, O=12.0 | próxima fase NS]
>> DECIDIR    A* encontró un plan de 11 acciones, demora 4.39 veh·h, 6366 nodos expandidos
>> ACTUAR     órdenes enviadas al controlador semafórico:
    1. verde NS 40 s  ->  [N=29.9, S=13.4, E=21.4, O=16.5 | próxima fase EO]
    2. verde EO 30 s  ->  [N=40.4, S=20.4, E=6.4, O=0.8 | próxima fase NS]
    3. verde NS 40 s  ->  [N=28.3, S=3.8, E=11.8, O=5.3 | próxima fase EO]
    4. verde EO 20 s  ->  [N=35.8, S=8.8, E=2.0, O=0.5 | próxima fase NS]
    5. verde NS 40 s  ->  [N=23.7, S=1.0, E=7.4, O=5.0 | próxima fase EO]
    6. verde EO 10 s  ->  [N=28.2, S=4.0, E=2.8, O=0.5 | próxima fase NS]
    7. verde NS 40 s  ->  [N=16.1, S=1.0, E=8.2, O=5.0 | próxima fase EO]
    8. verde EO 10 s  ->  [N=20.6, S=4.0, E=3.6, O=0.5 | próxima fase NS]
    9. verde NS 40 s  ->  [N=8.5, S=1.0, E=9.0, O=5.0 | próxima fase EO]
   10. verde EO 10 s  ->  [N=13.0, S=4.0, E=4.4, O=0.5 | próxima fase NS]
   11. verde NS 20 s  ->  [N=7.7, S=1.0, E=7.4, O=3.0 | próxima fase EO]
   objetivo alcanzado: True
```

Anexo C

Código Fuente de las Funciones Centrales del Modelo

El código completo se entrega junto con este documento. Se reproducen la función de transición con su costo y la heurística.

```
    def transicion(self, s: Estado, a: Accion):
        """Devuelve (estado_siguiente, costo) o (None, None) si hay desborde.

        Costo = demora total en vehículo-segundo = área bajo la curva de cola
        de los 4 accesos durante la acción (verde + tiempo perdido).
        """
        servidos = FASES[a.fase]
        g, L = a.verde, self.p.perdido
        nuevas, area = [], 0.0
        for i, q in enumerate(s.colas):
            lam = self._lam[i]
            if i in servidos:                       # acceso con verde
                r = self._sat - lam
                t0 = q / r                           # instante en que la cola se vacía
                if t0 >= g:
                    area += (q + (q - r * g)) * g / 2
                    q = q - r * g
                else:                                # se vacía y queda en 0
                    area += q * t0 / 2
                    q = 0
            else:                                    # acceso en rojo
                area += q * g + lam * g * g / 2
                q = q + lam * g
            # tiempo perdido (ámbar + todo rojo): todos los accesos acumulan
            area += q * L + lam * L * L / 2
            q = q + lam * L
            if q > self.p.cap:
                return None, None                    # desborde: acción inválida
            nuevas.append(int(q))
        return Estado(tuple(nuevas), SIGUIENTE[a.fase]), area / U   # veh·s

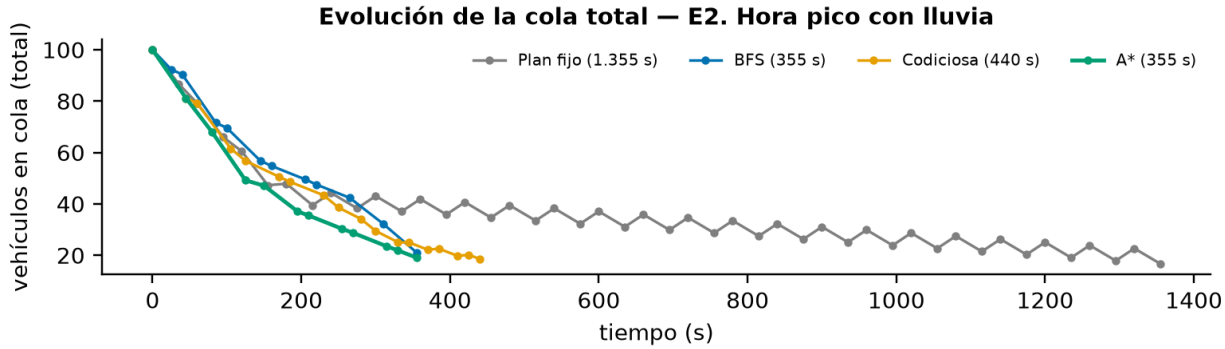
    def h(self, s: Estado) -> float:
        """Cota inferior de la demora que falta (veh·s).

        Ninguna cola puede bajar más rápido que r = s - lambda y la búsqueda no
        termina hasta que cada cola q llegue a la meta m. Por eso el área bajo
        su curva es, como mínimo, la del triángulo/trapecio que baja de q a m a
        tasa r:  (q^2 - m^2) / (2 r).  Sumar los 4 accesos sigue siendo una
        cota inferior (cada acceso aporta su propia área al costo), luego h es
        ADMISIBLE; además es consistente porque cada acción real tarda al menos
        lo que tarda la recta ideal.
        """
        m = self.p.q_meta
        total = 0.0
        for q, r in zip(s.colas, self._r):
            if q > m:
                total += (q * q - m * m) / (2 * r)
        return total / U
```

Anexo D
Evolución de las Colas con Cada Plan

Figura D1

Evolución de la cola total con cada plan en el escenario con lluvia



Nota. Cada punto es un cambio de fase; entre paréntesis, el tiempo hasta despejar. La meta exige que cada acceso tenga como máximo 8 veh, no solo la suma.